1 Introducción a la gestión de la ingeniería de software

Semana 1 · Resumen de la semana

Ideas clave, lección por lección

1 Qué es la ingeniería de software

- La ingeniería de software trata de entregar y mantener software, no solo de escribirlo.
- La mayoría de los problemas de proyecto vienen de brechas de comunicación y planificación.
- Los modelos permiten discutir y verificar un sistema antes de construirlo.

2 El ciclo de vida de desarrollo y los modelos de proceso

- Un modelo de proceso es una descripción simplificada de cómo trabajan los equipos reales.
- El trabajo iterativo produce líneas base; la diferencia entre ellas es un incremento.
- Elige un proceso según cuánto cambio esperas y cuán costosos son los errores tardíos.

3 Gestión de la configuración del software

- Los elementos de configuración incluyen todo lo que forma el producto, no solo el código.
- La GCS responde: qué versión, qué cambio, aprobado por quién y en qué línea base.

4 Control de versiones

- Un VCS guarda el historial, permite recuperar versiones y muestra quién cambió qué y cuándo.
- Git es distribuido; GitHub es un servicio de alojamiento para repositorios Git.
- Un commit debe contener un solo cambio lógico con un mensaje claro.

5 Fundamentos de Git: áreas, commits y ramas

- Edita en el directorio de trabajo, prepara con git add y registra con git commit.
- git status y git log muestran dónde estás; úsalos antes y después de cada paso.
- Una rama es un puntero. Fusionar combina historiales; los conflictos requieren una decisión humana.

6 Alojamiento Git, remotos y flujos de colaboración

- origin es el nombre habitual del remoto que clonaste o conectaste.
- fetch descarga; pull descarga e integra; push sube.
- Las solicitudes de cambio hacen de la revisión un paso planificado antes de llegar a main.

Cuidado con esto

Error	Solución
Confundir Git con GitHub.	Git es la herramienta de tu equipo; GitHub aloja repositorios.

Olvidar leer git status antes de hacer commit.	Haz de git status un hábito antes y después de cada commit.
Creer que "compila" significa "cumple la necesidad".	Compara el resultado con requisitos escritos y pruebas.
Llamar "ágil" a un proyecto en cascada solo porque tiene reuniones.	Pregunta si la retroalimentación cambia el plan durante el proyecto.
Escribir mensajes como "cosas" o "cambios".	Describe el cambio en una línea imperativa de unos 50 caracteres.
Hacer commit sin revisar git status, incluyendo archivos no relacionados.	Ejecuta git status y git diff --staged antes de cada commit.
Borrar los marcadores de conflicto sin elegir el contenido correcto.	Decide el contenido final, quita las tres líneas de marcador, luego prepara y haz commit.
Subir cambios directamente a main en un proyecto compartido.	Trabaja en una rama de funcionalidad y abre una solicitud de cambio para revisión.